5.1 Pablo DEX 고급 기능 분석

개요

Pablo DEX는 Composable Finance의 핵심 탈중앙화 거래소로, Balancer V1 스타일의 가중치 기반 AMM(Automated Market Maker)을 구현합니다. 이 문서는 Pablo DEX의 고급 기능들을 심층 분석하며, 다양한 풀 타입, TWAP 오라클 시스템, 수수료 구조, LP 토큰 메커니즘을 다룹니다.

5.1.1 다양한 풀 타입 구현 분석

Dual Asset Constant Product Pool

Pablo DEX의 핵심은 `DualAssetConstantProduct` 구현으로, Balancer V1의 가중치 기반 상수곱 공식을 사용합니다.

핵심 수학 공식

```
// Balancer V1 가중치 기반 상수곱 공식
// (Balance_i / Weight_i)^Weight_i = k (상수)
// 여기서 i는 각 자산을 의미

pub fn compute_out_given_in(
balance_in: u128,
weight_in: Permill,
balance_out: u128,
weight_out: Permill,
```

```
amount_in: u128,

swap_fee: Permill,

) -> Result<u128, ArithmeticError>
```

가중치 기반 풀 생성

```
impl<T: Config> DualAssetConstantProduct<T> {

pub(crate) fn  do_create_pool(

who: &T::AccountId,

fee_config: FeeConfig,

assets_weights: BoundedBTreeMap<T::AssetId, Permill, ConstU32<2>>,

lp_token_id: Option<AssetIdOf<T>>,

) -> Result<(T::PoolId, AssetIdOf<T>), DispatchError> {

// 정확히 2개 자산 검증

ensure!(assets_weights.len() == 2, Error::<T>::InvalidPair);

// 가중치가 0이 아님을 검증

ensure!(assets_weights.values().non_zero_weights(), Error::<T>::WeightsMustBeNon

// 가중치 합이 1인지 검증 (100%)

ensure!(

assets_weights

.values()

.sum_weights()

.map(|total_weight| total_weight.is_one())
```

```
.unwrap_or(false),
Error::<T>::WeightsMustSumToOne
);
// 수수료율이 100% 미만인지 검증
ensure!(fee_config.fee_rate < Permill::one(), Error::<T>::InvalidFees);
}
}
```

지원되는 풀 비율

Pablo DEX는 다양한 가중치 비율의 풀을 지원합니다:

- **50:50 풀**: 전통적인 Uniswap 스타일 균등 가중치
- **80:20 풀**: 한 자산에 더 많은 가중치 (거버넌스 토큰 등에 유용)
- **60:40 풀**: 중간 수준의 비대칭 가중치
- **기타 비율**: 가중치 합이 100%인 모든 조합 지원

가중치 기반 스왑 계산

```
pub(crate) fn  get_exchange_value(
pool: &BasicPoolInfo<T::AccountId, T::AssetId, ConstU32<2>>,
pool_account: &T::AccountId,
in_asset: AssetAmount<T::AssetId, T::Balance>,
out_asset_id: T::AssetId,
apply_fees: bool,
) -> Result<(AssetAmount<T::AssetId, T::Balance>, AssetAmount<T::AssetId, T::Bal
```

```
let pool_balances = Self::get_pool_balances(pool, pool_account);
// 입력 자산과 출력 자산의 가중치 및 잔액 조회
let (in_weight, in_balance) = pool_balances.get(&in_asset.asset_id)
.ok_or(Error::<T>::AssetNotFound)?;
let (out_weight, out_balance) = pool_balances.get(&out_asset_id)
.ok_or(Error::<T>::AssetNotFound)?;
// Balancer V1 공식을 사용한 출력량 계산
let amount_out = compute_out_given_in(
*in_balance,
*in_weight,
*out_balance,
*out_weight,
T::Convert::convert(in_asset.amount),
if apply_fees { pool.fee_config.fee_rate } else { Permill::zero() },
)?;
}
```

향후 확장 가능한 풀 타입

코드에서 LBP(Liquidity Bootstrapping Pool) 관련 에러 타입들이 발견됩니다:

```
#[pallet::error]
pub enum Error<T> {
```

```
// ... 기타 에러들
NoLpTokenForLbp,
NoXTokenForLbp,
// ...
}
```

이는 향후 LBP 기능이 추가될 예정임을 시사합니다.

5.1.2 TWAP (Time-Weighted Average Price) 계산 및 가격 오라클 기능

TWAP 시스템 아키텍처

Pablo DEX는 정교한 TWAP 시스템을 구현하여 조작 저항성이 있는 가격 오라클을 제공합니다.

TWAP 데이터 구조

```rust
#[derive(Encode, Decode, TypeInfo, Clone, Default, PartialEq, Eq, RuntimeDebug)]
pub struct TimeWeightedAveragePrice<Timestamp, Balance> {
pub timestamp: Timestamp, // 마지막 업데이트 시간
pub base_price_cumulative: Balance, // 기준 자산 누적 가격
pub quote_price_cumulative: Balance,// 상대 자산 누적 가격
pub base_twap: Rate, // 기준 자산 TWAP
pub quote_twap: Rate, // 상대 자산 TWAP
}
```

```
#[derive(Encode, Decode, TypeInfo, Clone, Default, PartialEq, Eq, RuntimeDebug)]
pub struct PriceCumulative<Timestamp, Balance> {
pub timestamp: Timestamp,
pub base_price_cumulative: Balance,
pub quote_price_cumulative: Balance,
}
```

TWAP 활성화 및 초기화

```
#[pallet::call_index(5)]
#[pallet::weight(100_000)]
#[transactional]
pub fn enable_twap(origin: OriginFor<T>, pool_id: T::PoolId) -> DispatchResult
let _who = ensure_signed(origin)?;
let pool = Self::get_pool(pool_id)?;
// 현재 가격을 기준으로 초기 누적 가격 상태 설정
let current_timestamp = T::Time::now();
let rate_base = Self::do_get_exchange_rate(pool_id, PriceRatio::NotSwapped)?;
let rate_quote = Self::do_get_exchange_rate(pool_id, PriceRatio::Swapped)?;
// 초기 누적 가격 상태 저장
PriceCumulativeState::<T>::insert(
pool_id,
PriceCumulative {
timestamp: current_timestamp,
```

```
base_price_cumulative: T::Balance::zero(),

quote_price_cumulative: T::Balance::zero(),

},

);

Ok(())

}
```

TWAP 계산 알고리즘

가격 누적 상태 업데이트

```
pub(crate) fn  update_price_cumulative_state<T: Config>(

pool_id: T::PoolId,

prev_price_cumulative: &mut Option<PriceCumulativeStateOf<T>>,

) -> Result<(T::Balance, T::Balance), DispatchError> {

if  let Some(previous_price_cumulative) = prev_price_cumulative {

let  current_timestamp = T::Time::now();

// 현재 가격 조회

let  rate_base = crate::Pallet::<T>::do_get_exchange_rate(pool_id, PriceRatio::N

let  rate_quote = crate::Pallet::<T>::do_get_exchange_rate(pool_id, PriceRatio::

// 누적 가격 계산: cumulative_price += current_price * time_elapsed

let (base_price_cumulative, quote_price_cumulative) = (

compute_next_price_cumulative::<T::Convert, _, _>(

previous_price_cumulative.timestamp,
```

```
previous_price_cumulative.base_price_cumulative,

current_timestamp,

rate_base,

)?,

compute_next_price_cumulative::<T::Convert, _, _>(

previous_price_cumulative.timestamp,

previous_price_cumulative.quote_price_cumulative,

current_timestamp,

rate_quote,

)?,

);

// 상태 업데이트

*prev_price_cumulative = Some(PriceCumulative {

timestamp: current_timestamp,

base_price_cumulative,

quote_price_cumulative,

});

Ok((base_price_cumulative, quote_price_cumulative))

} else {

Ok((T::Balance::zero(), T::Balance::zero()))

}

}
```

TWAP 값 계산

```
pub(crate) fn update_twap_state<T: Config>(
base_price_cumulative: T::Balance,
quote_price_cumulative: T::Balance,
prev_twap_state: &mut Option<TWAPStateOf<T>>,
) -> Result<(), DispatchError> {
if let Some(previous_twap_state) = prev_twap_state {
let current_timestamp = T::Time::now();
let elapsed = current_timestamp.saturating_sub(previous_twap_state.timestamp);
// TWAP 간격 조건 확인
if elapsed >= T::TWAPInterval::get() {
// TWAP 계산: (current_cumulative - previous_cumulative) / elapsed_time
let (average_price_base, average_price_quote) = get_twap_price::<T>(
base_price_cumulative,
previous_twap_state.base_price_cumulative,
quote_price_cumulative,
previous_twap_state.quote_price_cumulative,
elapsed,
)?;
// TWAP 상태 업데이트
*prev_twap_state = Some(TimeWeightedAveragePrice {
base_price_cumulative,
```

```
quote_price_cumulative,
timestamp: current_timestamp,
base_twap: average_price_base,
quote_twap: average_price_quote,
});
// TWAP 업데이트 이벤트 발생
Self::deposit_event(Event::TwapUpdated {
pool_id,
timestamp: current_timestamp,
twaps: BTreeMap::from([
(base_asset_id, average_price_base),
(quote_asset_id, average_price_quote),
]),
});
}
}
Ok(())
}
```

블록 훅을 통한 자동 TWAP 업데이트

```
#[pallet::hooks]
impl<T: Config> Hooks<T::BlockNumber> for Pallet<T> {
```

```
fn on_initialize(_block_number: T::BlockNumber) -> Weight {
let mut total_weight = Weight::zero();
// TWAP가 활성화된 모든 풀에 대해 업데이트 수행
let twap_enabled_pools: Vec<T::PoolId> = PriceCumulativeState::<T>::iter_keys()
for pool_id in twap_enabled_pools {
// 가격 누적 상태 업데이트
if let Ok(mut price_cumulative) = PriceCumulativeState::<T>::try_get(pool_id)
if let Ok((base_cumulative, quote_cumulative)) =
update_price_cumulative_state::<T>(pool_id, &mut Some(price_cumulative.clone()))
// TWAP 상태 업데이트
if let Ok(mut twap_state) = TWAPState::<T>::try_get(pool_id) {
let _ = update_twap_state::<T>(
base_cumulative,
quote_cumulative,
&mut Some(twap_state.clone()),
);
TWAPState::<T>::insert(pool_id, twap_state);
}
PriceCumulativeState::<T>::insert(pool_id, price_cumulative);
}
}
total_weight = total_weight.saturating_add(T::WeightInfo::update_twap());
}
```

```
total_weight

}

}
```

TWAP 조작 저항성

TWAP 시스템은 다음과 같은 방법으로 가격 조작을 방지합니다:

1. **시간 가중 평균**: 단순 평균이 아닌 시간 가중 평균 사용
2. **최소 간격 요구**: `TWAPInterval` 이상의 시간이 경과해야 업데이트
3. **누적 가격 사용**: 순간 가격이 아닌 누적 가격 기반 계산
4. **블록 단위 업데이트**: 매 블록마다 자동으로 가격 누적

5.1.3 수수료 구조 및 LP 토큰 발행/소각 메커니즘

수수료 구조 분석

Pablo DEX는 3단계 수수료 구조를 사용합니다:

```
pub struct FeeConfig {

pub fee_rate: Permill, // 총 수수료율

pub owner_fee_rate: Permill, // 풀 소유자 수수료율

pub protocol_fee_rate: Permill, // 프로토콜 수수료율

}


impl FeeConfig {
```

```
pub fn default_from(fee: Permill) -> Self {
Self {
fee_rate: fee,
owner_fee_rate: Permill::zero(),
protocol_fee_rate: Permill::zero(),
}
}
}
```

수수료 계산 및 분배

```
#[transactional]
fn disburse_fees(
who: &T::AccountId,
_: &T::PoolId,
owner: &T::AccountId,
fees: &Fee<T::AssetId, T::Balance>,
) -> Result<(), DispatchError> {
for (asset_id, fee_amount) in fees.iter() {
if !fee_amount.is_zero() {
// 풀 소유자에게 수수료 지급
T::Assets::transfer(
*asset_id,
who,
```

```
owner,
*fee_amount,
Preservation::Expendable,
)?;
}
}
Ok(())
}
```

LP 토큰 발행 메커니즘

초기 유동성 제공 시 LP 토큰 발행

```
// 첫 번째 유동성 제공 시
if  lp_total_issuance.is_zero() {
let  lp_to_mint = compute_first_deposit_lp(
assets_with_balances
.iter()
.map(|adi| (adi.asset_id, adi.deposit_amount, adi.asset_weight)),
Permill::zero(), // 초기 예치 시 수수료 없음
)?
.value;
// LP 토큰 발행
T::Assets::mint_into(pool.lp_token, who, T::Convert::convert(lp_to_mint))?;
```

```
}
```

추가 유동성 제공 시 LP 토큰 발행

```
// 단일 자산 예치 시
let single_deposit = compute_deposit_lp(
lp_total_issuance, // 현재 LP 토큰 총 발행량
single.deposit_amount, // 예치할 자산 수량
single.existing_balance, // 풀 내 기존 자산 잔액
single.asset_weight, // 자산 가중치
pool.fee_config.fee_rate, // 수수료율
)?;

// LP 토큰 발행
T::Assets::mint_into(
pool.lp_token,
who,
T::Convert::convert(single_deposit.value)
)?;
```

LP 토큰 발행량 계산 공식

```
// composable_maths::dex::constant_product에서
pub fn compute_deposit_lp(
```

```
lp_total_issuance: u128,

deposit_amount: u128,

existing_balance: u128,

asset_weight: Permill,

swap_fee: Permill,

) -> Result<ComputeDepositLpResult, ArithmeticError> {

// 가중치 기반 LP 토큰 발행량 계산

// lp_amount = lp_total_issuance * (deposit_amount / existing_balance) * asset_w

// 수수료 적용 시 발행량 조정

}
```

LP 토큰 소각 메커니즘

유동성 제거 시 LP 토큰 소각

```
pub(crate) fn  remove_liquidity(

who: &T::AccountId,

pool: BasicPoolInfo<T::AccountId, T::AssetId, ConstU32<2>>,

pool_account: T::AccountId,

lp_amount: T::Balance,

min_receive: BTreeMap<T::AssetId, T::Balance>,

) -> Result<BTreeMap<T::AssetId, T::Balance>, DispatchError> {

let  lp_total_issuance = T::Convert::convert(T::Assets::total_issuance(pool.lp_t

// 각 자산별 상환 가능 수량 계산
```

```
let mut redeemable_assets = BTreeMap::new();
for (asset_id, weight) in pool.assets_weights.iter() {
let asset_balance = T::Convert::convert(T::Assets::balance(*asset_id, &pool_acc
// LP 토큰 비율에 따른 자산 상환량 계산
let redeemable_amount = compute_redeemed_for_lp(
lp_total_issuance,
T::Convert::convert(lp_amount),
asset_balance,
*weight,
)?;
redeemable_assets.insert(*asset_id, T::Convert::convert(redeemable_amount.value)
}
// 최소 수령량 검증
for (asset_id, min_amount) in min_receive.iter() {
let redeemable = redeemable_assets.get(asset_id).ok_or(Error::<T>::AssetNotFoun
ensure!(*redeemable >= *min_amount, Error::<T>::CannotRespectMinimumRequested);
}
// LP 토큰 소각
T::Assets::burn_from(
pool.lp_token,
who,
lp_amount,
Precision::Exact,
```

```
Fortitude::Polite,
)?;
// 자산 전송
for (asset_id, amount) in  redeemable_assets.iter() {
T::Assets::transfer(
*asset_id,
&pool_account,
who,
*amount,
Preservation::Expendable,
)?;
}
Ok(redeemable_assets)
}
```

LP 토큰 가치 계산

실시간 LP 토큰 가치 조회

```
fn  redeemable_assets_for_lp_tokens(
pool_id: Self::PoolId,
lp_amount: Self::Balance,
) -> Result<BTreeMap<Self::AssetId, Self::Balance>, DispatchError> {
let  pool = Self::get_pool(pool_id)?;
```

```
let pool_account = Self::account_id(&pool_id);
match pool {
PoolConfiguration::DualAssetConstantProduct(info) => {
let lp_total_issuance = T::Convert::convert(T::Assets::total_issuance(info.lp_t
let mut redeemable = BTreeMap::new();
for (asset_id, weight) in info.assets_weights.iter() {
let balance = T::Convert::convert(T::Assets::balance(*asset_id, &pool_account))
let redeemable_amount = compute_redeemed_for_lp(
lp_total_issuance,
T::Convert::convert(lp_amount),
balance,
*weight,
)?;
redeemable.insert(*asset_id, T::Convert::convert(redeemable_amount.value));
}
Ok(redeemable)
}
}
}
```

5.1.4 고급 AMM 기능

시뮬레이션 기능

Pablo DEX는 실제 거래 실행 전 결과를 미리 볼 수 있는 시뮬레이션 기능을 제공합니다:

```
fn simulate_add_liquidity(
pool_id: Self::PoolId,
amounts: BTreeMap<Self::AssetId, Self::Balance>,
) -> Result<Self::Balance, DispatchError> {
// 실제 상태 변경 없이 LP 토큰 발행량 계산
}

fn simulate_remove_liquidity(
pool_id: Self::PoolId,
lp_amount: Self::Balance,
) -> Result<BTreeMap<Self::AssetId, Self::Balance>, DispatchError> {
// 실제 상태 변경 없이 상환 가능 자산량 계산
}
```

슬리피지 보호

모든 거래에서 최소 수령량을 지정하여 슬리피지를 제한할 수 있습니다:

```
pub fn swap(
origin: OriginFor<T>,
pool_id: T::PoolId,
in_asset: AssetAmount<T::AssetId, T::Balance>,
min_receive: AssetAmount<T::AssetId, T::Balance>, // 최소 수령량
```

```
keep_alive: bool,
) -> DispatchResult {
// 계산된 출력량이 최소 수령량보다 큰지 검증
ensure!(
amount_out.amount >= min_receive.amount,
Error::<T>::CannotRespectMinimumRequested
);
}
```

가스 효율성 최적화

- **배치 TWAP 업데이트**: 모든 풀의 TWAP를 한 번에 업데이트
- **지연 계산**: 필요할 때만 복잡한 계산 수행
- **스토리지 최적화**: 효율적인 스토리지 레이아웃 사용

5.1.5 보안 및 안전성

트랜잭션 원자성

모든 중요한 함수는 `#[transactional]` 매크로를 사용하여 원자성을 보장합니다:

```
#[transactional]
fn do_swap(...) -> Result<SwapResult<...>, DispatchError> {
// 모든 작업이 성공하거나 모두 롤백
}
```

입력 검증

- **자산 존재성 검증**: 풀에 포함된 자산인지 확인
- **수량 유효성 검증**: 0이 아닌 양수인지 확인
- **권한 검증**: 적절한 권한을 가진 계정인지 확인
- **슬리피지 검증**: 최소 수령량 조건 만족 여부 확인

오버플로우 방지

모든 수학적 계산에서 안전한 연산을 사용합니다:

```
use composable_support::math::safe::SafeAdd;
```

```
*pool_count = pool_id.safe_add(&T::PoolId::one())?;
```

이러한 고급 기능들을 통해 Pablo DEX는 안전하고 효율적이며 사용자 친화적인 탈중앙화 거래소 서비스를 제공합니다.